

Shape Bounding Box Calculator

Yusupov Boburjon

Neptun: YTAJDI

Course: Programming Technology

October 6, 2025

Abstract

This document describes a small Java project that loads several regular shapes from a text file (circle, regular triangle, square, regular hexagon), computes their axis-aligned bounding boxes, and determines which shape has the greatest bounding box area. The implementation uses an object-oriented design with a common abstract superclass, validated input, exception handling, and stores shapes in a single collection processed with `foreach`. The document contains a concise description of the exercise, a class diagram, short method descriptions, and black-box/white-box test cases.

1 Problem description

Fill a collection with several regular shapes (circle, regular triangle, square, regular hexagon). Which shape has the greatest bounding box area?

A bounding box of a shape covers the shape completely, and its sides are parallel with the x or y axis. Each shape can be represented by its center and side length (or radius), if we assume that one side of the polygons are parallel with x axis, and its nodes lies on or above this side. Load and create the shapes from a text file. The first line of the file contains the number of the shapes, and each following line contains a shape. The first character will identify the type of the shape, which is followed by the center coordinate and the side length or radius. Manage the shapes uniformly, so derive them from the same super class.

2 Design overview

2.1 Classes

The project contains the following domain classes:

- **Shape** (abstract): common superclass. Stores center coordinates and defines an abstract method `getBoundingBoxArea()`.
- **Circle** extends **Shape**: stores `radius`, computes bounding box area as $(2r)^2$.
- **Square** extends **Shape**: stores `side`, bounding box area = $side^2$.
- **RegularTriangle** extends **Shape**: stores `side`. With a flat base at the bottom, bounding box width = `side`, height = $\frac{\sqrt{3}}{2} side$, so area = $side \cdot \frac{\sqrt{3}}{2} side$.
- **RegularHexagon** extends **Shape**: stores `side`. For a flat-topped orientation, bounding box width = $2 \cdot side$, height = $\sqrt{3} \cdot side$, so area = $2 \cdot side \cdot \sqrt{3} \cdot side$.
- **InvalidShapeDataException**: custom exception used when constructor validation fails (e.g. non-positive size).

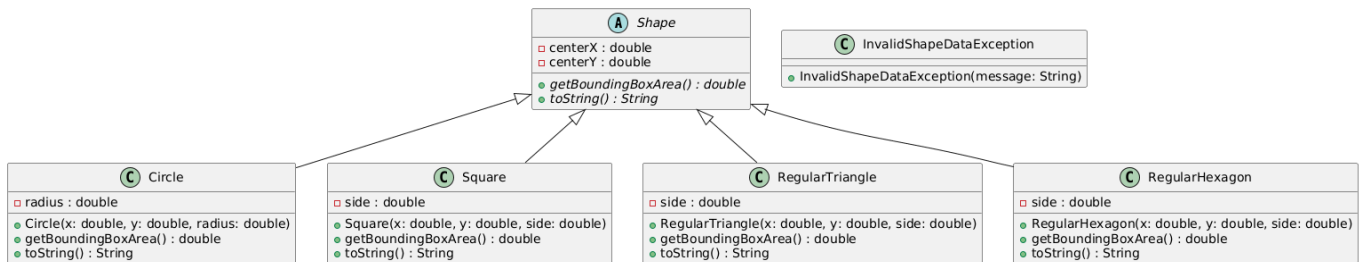
2.2 Collection and processing

All shape instances are stored in a `List<Shape>` (e.g. `ArrayList<Shape>`). The program reads shapes from the input file, validates data, creates objects, and adds them to this collection. A `foreach` loop is used to compute bounding box areas and to find the maximum.

2.3 Validation and exceptions

Constructors validate their parameters; if invalid, they throw `InvalidShapeDataException`. The reader code (testing / main) catches and handles these exceptions so invalid lines are skipped without terminating the program.

3 Class diagram



4 Short description of methods and constructors

Below are concise descriptions for the most important methods (useful for documentation / javadoc).

4.1 Shape (abstract)

- `protected Shape(double x, double y)`: constructor, stores center coordinates.
- `public abstract double getBoundingBoxArea()`: returns bounding box area for the concrete shape.
- `public double getCenterX(), getCenterY()`: accessors for testing.

4.2 Circle

- `public Circle(double x, double y, double radius)` throws `InvalidShapeDataException`: validates `radius > 0`, stores values.
- `public double getBoundingBoxArea()`: returns $(2 \cdot radius)^2$.
- `public double getRadius()`: accessor for tests.
- `public String toString()`: human-readable representation.

4.3 Square

- `public Square(double x, double y, double side)` throws `InvalidShapeDataException`: validates `side > 0`.
- `public double getBoundingBoxArea()`: returns `side * side`.
- `public double getSide()`: accessor.

4.4 RegularTriangle

- `public RegularTriangle(double x, double y, double side)` throws `InvalidShapeDataException` validates `side > 0`.
- `public double getBoundingBoxArea():` returns `side * (sqrt(3)/2 * side)`.
- `public double getSide():` accessor.

4.5 RegularHexagon

- `public RegularHexagon(double x, double y, double side)` throws `InvalidShapeDataException` validates `side > 0`.
- `public double getBoundingBoxArea():` returns `(2 * side) * (sqrt(3) * side)`.
- `public double getSide():` accessor.

4.6 InvalidShapeDataException

- `public InvalidShapeDataException(String message):` constructs the validation exception.

5 Testing

This section contains representative black-box and white-box test cases. I used JUnit 4 as a primary testing framework.

5.1 Black-box test cases (behavioral)

Black-box tests treat the class as a black box: they assert output or thrown exceptions for given inputs.

```
1  @Test
2  public void testValidCircleBoundingBoxArea() throws
    InvalidShapeDataException {
3      Circle c = new Circle(0,0,5);
4      assertEquals(100.0, c.getBoundingBoxArea(), 1e-4);
5  }
6
7  @Test
8  public void testInvalidCircleRadiusThrows() {
9      assertThrows(InvalidShapeDataException.class, () -> new Circle(0,0,-1)
10         );
11  }
12
13 @Test
14 public void testValidTriangleBoundingBoxArea() throws
15     InvalidShapeDataException {
16     RegularTriangle t = new RegularTriangle(0,0,4);
17     double expected = 4 * ((Math.sqrt(3)/2) * 4);
18     assertEquals(expected, t.getBoundingBoxArea(), 1e-4);
19 }
20
21 @Test
22 public void testValidSquareBoundingBoxArea() throws
23     InvalidShapeDataException {
24     Square s = new Square(0,0,5);
25     assertEquals(25.0, s.getBoundingBoxArea(), 1e-4);
26 }
27
28 @Test
29 public void testValidHexagonBoundingBoxArea() throws
30     InvalidShapeDataException {
31     RegularHexagon h = new RegularHexagon(0,0,3);
32     double expected = (2*3) * (Math.sqrt(3) * 3);
33     assertEquals(expected, h.getBoundingBoxArea(), 1e-4);
34 }
```

Listing 1: Black-box tests (JUnit 4 snippets)

5.2 White-box test cases (structural)

White-box tests check internal logic, branches, and edge cases. They may require accessors to inspect stored fields.

```
1  @Test
2  public void testCircleConstructorStoresValues() throws
    InvalidShapeDataException {
3      Circle c = new Circle(1.5, -2.5, 3.0);
4      assertEquals(3.0, c.getRadius(), 1e-4);
5      assertEquals(1.5, c.getCenterX(), 1e-4);
6      assertEquals(-2.5, c.getCenterY(), 1e-4);
7  }
8
9  @Test
10 public void testTriangleZeroSideThrows() {
11     assertThrows(InvalidShapeDataException.class, () -> new
        RegularTriangle(0,0,0));
12 }
13
14 @Test
15 public void testHexagonAreaPrecision() throws InvalidShapeDataException {
16     RegularHexagon h = new RegularHexagon(0,0,1);
17     double expected = 2 * 1 * (Math.sqrt(3) * 1);
18     assertEquals(expected, h.getBoundingBoxArea(), 1e-9);
19 }
20
21 @Test
22 public void testSquareZeroSideThrows() {
23     assertThrows(InvalidShapeDataException.class, () -> new Square(0,0,0))
        ;
24 }
```

Listing 2: White-box tests (JUnit 4 snippets)

5.3 Test input file examples

Example data.txt content (first line = number of shapes):

```
5
C 0 0 5
S 1 1 4
T -2 3 6
H 0 0 3
C 2 2 2
```

6 Usage notes and implementation hints

- Read input using `BufferedReader`. Parse the first line as integer count. For each subsequent line split into tokens: `type x y size`.
- Validate: number of tokens, numeric parsing, `size > 0`, known type. On invalid line print a warning and skip the line.
- Store valid shapes in `List<Shape>` and use `for (Shape s : shapes)` to process them.

- To find the shape with the largest bounding box area, iterate and keep track of the maximum area and corresponding object.
- For small educational projects, making `InvalidShapeDataException` a `RuntimeException` simplifies code (no checked-throws declarations). If you prefer checked exceptions, declare `throws` on constructors and handle accordingly.

7 Conclusion

The presented design fulfills the requirements:

- Unified treatment of shapes via an abstract superclass.
- A single collection stores all shape objects and is processed via `foreach`.
- Input is validated and invalid data is handled via exceptions.
- Documentation includes problem description, class diagram (PlantUML source), short method descriptions, and example white-box/black-box tests.